

Rubik's Cube Solver — Complete Interview Prep

Rubik's Cube Solver — Complete Interview Preparation

Author: Shoaib Samim

Language: C++14

Build System: CMake

Core Algorithm: IDA* with Corner Pattern Database Heuristic

PART 1 — QUICK REFERENCE (Memorize These)

Fact	Value
Total Rubik's Cube states	43,252,003,274,489,856,000 ($\sim 4.3 \times 10^{19}$)
God's Number (max moves to solve any state)	20 (Half Turn Metric)
Corner configurations	88,179,840 (~ 88 million)
Corner permutations	$8! = $ 40,320
Corner orientations	$3^7 = $ 2,187
Edge configurations	$12! \times 2^{11} = $ 980,995,276,800
Database size (nibble-packed)	~ 44 MB
Number of legal moves	18 (6 faces $\times$ 3: quarter, prime, double)
Branching factor	18
18^{10} (without pruning)	$\sim 3.6 \times 10^{12}$
With IDA* + heuristic	depth-10 solved in < 1 second

PART 2 — DEMO SCRIPT (Practice This Word for Word)

Step 1 — Build and Run

```
cd build
cmake ..
make -j10

# Four run modes:
./rubiks_cube_solver                # random
    scramble + single-threaded solve
./rubiks_cube_solver --fast          # random
    scramble + multithreaded solve
./rubiks_cube_solver --scan ../Databases/cornerDepth8V1.txt #
    scan physical cube via webcam + solve
./rubiks_cube_solver --scan --fast ../Databases/cornerDepth8V1.txt #
    scan + multithreaded (recommended demo)
```

Step 2 — What to Say

“This project implements a Rubik’s Cube solver in C++ with three components: - The cube is represented as a **bitboard** — 6×64 -bit integers, one per face — so moves are just bit shifts. - The solver is **IDA*** with an admissible heuristic from a precomputed **corner pattern database** (44 MB, built via BFS from the solved state, storing minimum move count for all 88M corner configurations). - I added a **camera scanner** using OpenCV so a real physical cube can be scanned and solved, and a **multithreaded IDA*** variant that splits the 18 root moves across 10 worker threads for a $\sim 9\times$ speedup on my M4 Air.”

Step 3 — Point to Output

- **Scrambled cube print** → “This is the planar unfolding. U is top, L F R B is the middle row, D is bottom.”
- **Shuffle moves** → “These are the 10 random moves applied. Each letter is a face: U=Up, R=Right, F=Front, etc. Prime (') means counter-clockwise, 2 means 180 degrees.”
- **Solution moves** → “IDA* found this sequence that reverses the scramble.”
- **Time** → “Under a second for 10 moves. Deeper scrambles take longer because the search space grows exponentially.”

Step 4 — Anticipate the First Follow-Up

They will ask: “Why not just BFS?”

Answer: “BFS stores every visited state. At depth 10 alone, that’s $18^{10} \approx 3.6$ trillion states. Even at 10 bytes each, that’s 36 petabytes — impossible. IDA* uses $O(\text{depth})$ ”

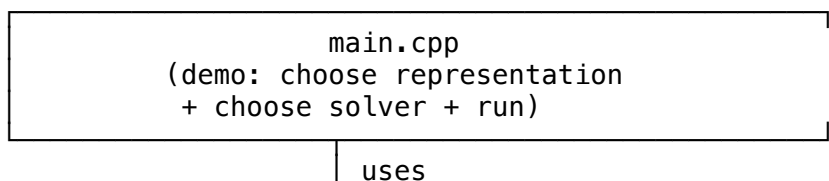
space by trading some time for space, and the heuristic prunes most of that time away anyway.”

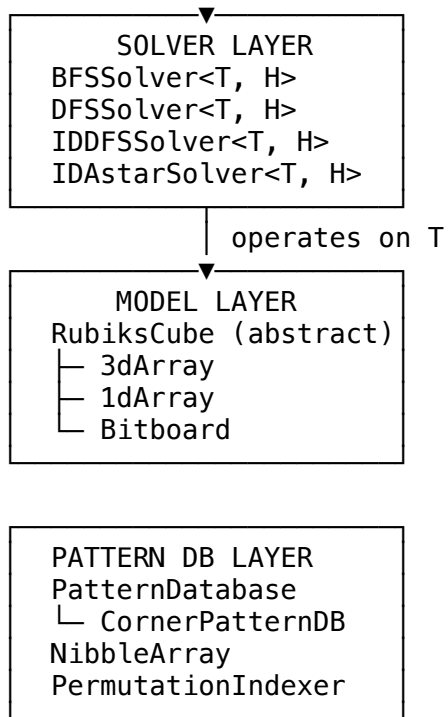
PART 3 — PROJECT ARCHITECTURE

Folder Structure

```
rubiks-cube-solver/
├── Model/
│   ├── RubiksCube.h          ← Abstract base class (interface)
│   └── RubiksCube.cpp        ← Shared methods: print(), move(),
invert()
│   ├── RubiksCube3dArray.cpp ← Representation 1: cube[6][3][3]
│   ├── RubiksCube1dArray.cpp ← Representation 2: cube[54]
(flattened)
│   └── RubiksCubeBitboard.cpp ← Representation 3: bitboard[6]
uint64_t
├── Solver/
│   ├── BFSSolver.h          ← Breadth-First Search solver
(template)
│   ├── DFSSolver.h          ← Depth-First Search solver (template)
│   ├── IDDFSolver.h         ← Iterative Deepening DFS solver
(template)
│   ├── IDAstarSolver.h      ← IDA* solver (single-threaded)
│   └── IDAstarSolverMT.h    ← IDA* solver (multithreaded, uses --
fast flag)
├── PatternDatabases/
│   ├── NibbleArray.h/.cpp   ← Memory-efficient 4-bit array
│   ├── PatternDatabase.h/.cpp ← Abstract base for databases
│   └── CornerPatternDatabase.h/.cpp ← Corner-specific DB (88M
entries)
│   ├── PermutationIndexer.h ← Lehmer code for permutation ranking
│   ├── CornerDBMaker.h/.cpp ← BFS to build and store corner DB
│   └── math.h/.cpp          ← Factorial/permutation math helpers
├── Scanner/
│   ├── CubeScanner.h        ← Camera scanning interface (uses
OpenCV)
│   └── CubeScanner.cpp      ← Live camera + click-to-fix UI
├── Databases/
│   └── cornerDepth8V1.txt    ← Precomputed binary database file
├── compat/
│   └── bits/stdc++.h        ← Compatibility shim (Apple Clang)
└── main.cpp                ← Demo: scramble / scan → solve → print
```

Three-Layer Architecture





Design Patterns Used

Pattern	Where	Why
Template Method	RubiksCube abstract class	Base defines the interface; subclasses implement moves
Strategy	Solver templates <T,H>	Swap algorithm without changing cube code
Compile-time Polymorphism	template<typename T, typename H>	Avoids vtable overhead in tight loops
Factory (informal)	main.cpp	Chooses which concrete cube to instantiate

PART 4 — RUBIK'S CUBE FUNDAMENTALS

Anatomy

- **6 faces:** Up (U), Down (D), Front (F), Back (B), Left (L), Right (R)

- **26 physical pieces:** 8 corners (3 stickers each), 12 edges (2 stickers each), 6 centres (1 sticker each, fixed)
- **54 stickers total**
- **Centres never move** relative to each other — they define face colors

The 18 Moves

Face moves $\times$ 3 variants each = $6 \times 3 = 18$ total

L	L'	L2	(Left face: clockwise, counter-clockwise, 180°)
R	R'	R2	(Right face)
U	U'	U2	(Up face)
D	D'	D2	(Down face)
F	F'	F2	(Front face)
B	B'	B2	(Back face)

State Space Calculation

Corner permutations: $8! = 40,320$
 Corner orientations: $3^8 / 3 = 3^7 = 2,187$
 (8th orientation determined by first 7 – parity constraint)

Edge permutations: $12! = 479,001,600$
 Edge orientations: $2^{12} / 2 = 2^{11} = 2,048$
 (12th orientation determined by first 11 – parity constraint)

Parity constraint: divide by 2
 (odd corner perm $\leftrightarrow$ odd edge perm – they must match)

Total = $(8! \times 3^7 \times 12! \times 2^{11}) / 2$
 $= 43,252,003,274,489,856,000$
 $\approx 4.3 \times 10^{19}$

Corner Encoding (5-bit)

Each corner is encoded as a 5-bit integer:

Bit 4: orientation (White/Yellow on which sticker position – 0=top, 1=front, 2=side)
 Bit 3: orientation bit 2
 Bit 2: 1 if bottom (Yellow), 0 if top (White)
 Bit 1: 1 if Orange face, 0 if Red face
 Bit 0: 1 if Green face, 0 if Blue face

So each corner is uniquely identified by which white/yellow + red/orange + blue/green face it shows.

PART 5 — CUBE REPRESENTATIONS

Representation 1: 3D Array

```
uint8_t cube[6][3][3];  
// cube[face][row][col]  
// face: 0=UP, 1=LEFT, 2=FRONT, 3=RIGHT, 4=BACK, 5=DOWN  
// values: 0=WHITE, 1=GREEN, 2=RED, 3=BLUE, 4=ORANGE, 5=YELLOW
```

Pros: Most readable, easy to debug, matches mental model

Cons: 54 bytes, poor cache locality for move operations (scattered memory accesses)

Representation 2: 1D Array (Flattened)

```
uint8_t cube[54];  
// Flattened version of 3D: index = face*9 + row*3 + col
```

Pros: Better cache locality than 3D, single contiguous block

Cons: Index arithmetic harder to read, still 54 bytes per state

Representation 3: Bitboard

```
uint64_t bitboard[6]; // one 64-bit integer per face
```

Layout of each uint64_t (8 positions × 8 bits each = 64 bits):

Face positions (viewed from front):

```
  0 | 1 | 2  
  7 | 8 | 3    ← position 8 is centre (always same color, encoded  
implicitly)  
  6 | 5 | 4
```

Each 8-bit slot encodes color as a bitmask:

```
bit 0 = White (color 0)  
bit 1 = Green (color 1)  
bit 2 = Red   (color 2)  
bit 3 = Blue  (color 3)  
bit 4 = Orange (color 4)  
bit 5 = Yellow (color 5)
```

Example: Green sticker at position 3 of face 2:

```
bitboard[2] has bits [31:24] = 0b00000010 = 0x02
```

Why this works for moves: - Face rotation = one 64-bit barrel shift: `side = (side << 16) | (side >> 48)` - Sticker transfer = bitwise AND + OR with masks - Color lookup = find which bit is set: `bit_pos = trailing_zeros(color_byte)`

Pros: Fastest representation, entire face in one CPU register, bitwise ops are single-cycle

Cons: Hardest to understand and debug, encoding is non-obvious

PART 6 — ALGORITHMS WITH PSEUDOCODE

Algorithm 1: BFS (Breadth-First Search)

Concept

Explore cube states level by level. Level 0 = initial state. Level 1 = all states reachable in 1 move. Level k = all states reachable in exactly k moves. First time we reach the solved state, we have the shortest path.

Pseudocode

```
BFS(initial_cube):
    queue Q
    visited = empty hash map
    move_done = empty hash map    ← back-pointer: state → move that
    reached it

    Q.push(initial_cube)
    visited[initial_cube] = true

    while Q is not empty:
        node = Q.pop_front()

        if node.isSolved():
            return reconstruct_path(node, move_done, initial_cube)

        for each move m in [L, L', L2, R, R', R2, U, U', U2, D, D',
D2, F, F', F2, B, B', B2]:
            node.apply(m)
            if node not in visited:
                visited[node] = true
                move_done[node] = m
                Q.push(node)
            node.undo(m)    ← invert the move to restore node

    return []    ← no solution (shouldn't happen for valid cube)

reconstruct_path(solved, move_done, initial):
    path = []
    curr = solved
    while curr != initial:
        m = move_done[curr]
        path.push_front(m)
        curr.undo(m)
    return path
```

Complexity

- **Time:** $O(18^d)$ where d = solution depth
- **Space:** $O(18^d)$ — stores every visited state in the hash map

- **Optimal:** YES — always finds shortest solution
- **Practical:** Only up to ~6 moves. At $d=7$, states exceed 100 million.

When BFS Fails

At depth 20 (God's Number), you'd need to store 43 quintillion states. Even at 1 byte per state, that's 43 exabytes — more than all data ever created by humans.

Algorithm 2: DFS (Depth-First Search)

Concept

Go deep down one path until you hit the depth limit or the solution. If neither, backtrack and try the next move.

Pseudocode

```
DFSsolver(cube, max_depth):
    moves_taken = []

    dfs(depth):
        if cube.isSolved():
            return true
        if depth > max_depth:
            return false

        for each move m in all 18 moves:
            cube.apply(m)
            moves_taken.push(m)

            if dfs(depth + 1):
                return true      ← found solution, unwind

            moves_taken.pop()
            cube.undo(m)          ← backtrack

        return false

    dfs(1)
    return moves_taken
```

Complexity

- **Time:** $O(18^{\text{max_depth}})$ — same as BFS in worst case
 - **Space:** $O(\text{max_depth})$ — only the current path on the stack
 - **Optimal:** NO — finds A solution, not necessarily shortest
 - **Weakness:** Without visited-state tracking, can revisit the same state millions of times (e.g., apply L, then L', then L, then L' — cycles forever unless depth cap hits first)
-

Algorithm 3: IDDFS (Iterative Deepening DFS)

Concept

Run DFS with depth limit 1. If no solution, run DFS with depth limit 2. Continue until solution found. Combines BFS optimality with DFS space efficiency.

Pseudocode

```
IDDFSolver(cube, max_depth):
    for depth_limit = 1 to max_depth:
        solver = DFSSolver(cube, depth_limit)
        result = solver.solve()
        if solver.cube.isSolved():
            return result
    return []
```

[Uses DFSSolver internally for each depth limit]

Why the Redundant Work is Small

At branching factor $b=18$, most nodes are at the deepest level:

```
Level 0: 1 node
Level 1: 18 nodes
Level 2:  $18^2 = 324$  nodes
...
Level d:  $18^d$  nodes
```

Total across all levels = $18^d \times (1 + 1/18 + 1/18^2 + \dots) \approx 18^d \times 18/17$

So IDDFS does only $18/17 \approx 1.06\times$ the work of a single DFS at depth d . The overhead is just 6% – negligible.

Complexity

- **Time:** $O(18^d)$ — same as BFS, small constant overhead
 - **Space:** $O(d)$ — only one stack path at a time
 - **Optimal:** YES — finds shortest solution (same guarantee as BFS)
-

Algorithm 4: IDA* (Iterative Deepening A*)

Concept

Like IDDFS, but instead of bounding by **depth alone**, bound by **$f = g + h$** , where: - g = moves taken so far (cost from start) - h = heuristic estimate of remaining moves (from pattern DB)

Only expand a node if $f \leq \text{current_bound}$. If f exceeds bound, record the minimum exceeded value as the next bound.

Pseudocode

```
IDASolver(cube, pattern_db):
    bound = h(cube)          ← initial bound = heuristic of start
    state

    while true:
        result = search(cube, 0, bound)
        if result == SOLVED:
            return reconstruct_path()
        if result == INFINITY:
            return []          ← no solution
        bound = result        ← tighten bound to smallest exceeded
                                f-value

search(node, g, bound):
    f = g + h(node)
    if f > bound:
        return f              ← exceeded bound, return f for next
                                iteration

    if node.isSolved():
        return SOLVED

    next_bound = INFINITY
    for each move m in all 18 moves:
        node.apply(m)
        if node not already visited:
            result = search(node, g+1, bound)
            if result == SOLVED:
                record_move(m)
                return SOLVED
            next_bound = min(next_bound, result)
        node.undo(m)

    return next_bound

h(node):
    return pattern_db.getNumMoves(node)  ← corner DB lookup
```

My Implementation (Bounded A* with Priority Queue)

The actual implementation uses a priority queue instead of pure recursion:

```
IDASolver(cube):
    pq = min-priority-queue ordered by (g + h)
    push (start_cube, depth=0, estimate=h(start_cube))
    next_bound = INFINITY

    while pq not empty:
        node = pq.pop()

        if visited[node]: continue
        visited[node] = true
        move_done[node] = move_that_reached_node
```

```

    if node.isSolved(): return (node, bound)

    for each move m in all 18 moves:
        node.apply(m)
        if not visited[node]:
            f = node.depth + h(node)
            if f > bound:
                next_bound = min(next_bound, f)
            else:
                pq.push(node, move=m)
        node.undo(m)

    return (original_cube, next_bound)

```

Note: This is technically bounded A*, not pure IDA*. Pure IDA* uses recursion and has $O(d)$ space. The PQ version uses $O(18^d)$ / pruning) space — worse but still heavily pruned.

Complexity

- **Time:** Depends heavily on heuristic quality. With perfect heuristic: $O(\text{solution length})$. In practice with corner DB: exponentially less than without.
- **Space:** $O(d)$ for pure recursive IDA*; $O(18^d)$ with PQ (but heavily pruned)
- **Optimal:** YES — admissible heuristic guarantees optimal solution
- **Practical:** Solves arbitrary scrambles with good heuristic in seconds

Why IDA* Beats A*

Property	A*	IDA*
Space	$O(b^d)$ — stores all frontier	$O(d)$ — just current path
Time	$O(b^d)$	$O(b^d)$ with small constant
Memory on 43Q states	Impossible	Fine (only path in memory)

PART 7 — DATA STRUCTURES IN DEPTH

7.1 NibbleArray

What It Is

An array that stores two 4-bit values per byte, halving memory compared to a `uint8_t` array.

Why It Matters

- Corner DB has ~88 million entries
- Move count fits in 0–15 (4 bits is enough)
- `uint8_t[]` would use 88 MB
- `NibbleArray` uses 44 MB — fits better in RAM cache

Internal Layout

Index 0 → upper nibble (bits 7:4) of `arr[0]`

Index 1 → lower nibble (bits 3:0) of `arr[0]`

Index 2 → upper nibble of `arr[1]`

Index 3 → lower nibble of `arr[1]`

...

`arr[i/2]` holds indices `i` and `i+1`

`GET(pos):`

`byte = arr[pos/2]`

 if `pos` is even: return `byte >> 4` (upper 4 bits)

 if `pos` is odd: return `byte & 0x0F` (lower 4 bits)

`SET(pos, val):`

`byte = arr[pos/2]`

 if `pos` is even: `arr[pos/2] = (byte & 0x0F) | (val << 4)`

 if `pos` is odd: `arr[pos/2] = (byte & 0xF0) | (val & 0x0F)`

Example

`pos=0, val=7: arr[0] = 0b01110000 = 0x70`

`pos=1, val=3: arr[0] = 0b01110011 = 0x73`

`get(0) = 0x73 >> 4 = 7 ✓`

`get(1) = 0x73 & 0x0F = 3 ✓`

7.2 PatternDatabase (Abstract Base)

What It Is

Abstract class that wraps a `NibbleArray` and provides move-count storage and file I/O.

Key Methods

`setNumMoves(cube, n)` → convert `cube` to DB index, store `n` (if `n <` current value)

`getNumMoves(cube)` → convert `cube` to DB index, return stored value

`toFile(path)` → write raw bytes of `NibbleArray` to binary file

`fromFile(path)` → read binary file back into `NibbleArray`

`getDatabaseIndex()` → PURE VIRTUAL – subclass computes the index

File I/O

Uses binary read/write on the raw byte array:

```
// Write:
writer.write(reinterpret_cast<const char*>(database.data()),
             database.storageSize());

// Read:
reader.read(reinterpret_cast<char*>(database.data()),
            database.storageSize());
```

`reinterpret_cast` treats the `uint8_t*` as `char*` for stream I/O — safe because `char` and `uint8_t` have the same size.

7.3 CornerPatternDatabase

What It Is

Concrete subclass of `PatternDatabase`. Computes a unique index for any cube's corner state.

Size

Permutations: $8! = 40,320$
Orientations: $3^7 = 2,187$
Total: $40,320 \times 2,187 = 88,179,840$

getDatabaseIndex — How It Works

Step 1: Extract corner permutation

For each of 8 corners (0–7), call `getCornerIndex(i)`

This identifies WHICH corner is at position `i`

(White/Yellow face bit + Red/Orange face bit + Blue/Green face bit)

Returns a value 0–7 identifying the corner

Example: corner index = `0b101` → Yellow + Red + Green

corners array = `[3, 0, 7, 1, 2, 5, 6, 4]`

Step 2: Rank the permutation using Lehmer code

`rank([3,0,7,1,2,5,6,4])` → single integer 0 to 40319

Step 3: Extract corner orientations

For each of first 7 corners, call `getCornerOrientation(i)`

Returns 0 (white/yellow on top), 1 (on front), or 2 (on side)

The 8th is always determined by the other 7 (parity)

`orientations = [0, 2, 1, 0, 2, 0, 1]`

Step 4: Compute orientation number (base-3)

`orientationNum = 0×36 + 2×35 + 1×34 + 0×33 + 2×32 + 0×3 + 1`
`= 0 + 486 + 81 + 0 + 18 + 0 + 1 = 586`

Step 5: Final index

```
index = rank × 2187 + orientationNum  
       = (rank of perm) × 2187 + 586
```

7.4 PermutationIndexer (Lehmer Code)

What It Is

Converts a permutation array like [3, 0, 7, 1, 2, 5, 6, 4] into a unique integer 0 to (N!-1) in O(N) time.

The Lehmer Code Idea

For permutation [3, 0, 7, 1, 2, 5, 6, 4]: - Position 0: digit 3. How many digits to its right are smaller? That's the Lehmer digit. - Position 1: digit 0. How many of the **remaining** digits to its right are smaller? - ...

This gives a number in the **factorial number system** (factoradic):

```
lehmer[0] × 7! + lehmer[1] × 6! + ... + lehmer[7] × 0!
```

Why O(N) with Bitset

Naively, counting “how many remaining digits are smaller” takes O(N²). The trick: - Maintain a bitset<N> of “seen” digits (marked from right) - For digit d, count the 1-bits to the RIGHT of position d in the bitset = number of remaining digits smaller than d - Bit-counting is O(1) with hardware popcount / lookup table

Bitset is indexed right-to-left:

After seeing digits 3 and 0:

bitset = ...10001 (bit N-1-0 and bit N-1-3 are set)

For digit 7: count 1s in bitset >> (N - 7)

This gives exactly how many "seen" digits are less than 7

(those seen digits are already placed, so 7 is effectively at index 7 - numOnes)

Precomputed Tables

```
onesCountLookup[i] = popcount(i)    ← precomputed for all i from 0 to 2^N
```

```
factorials[i] = P(N-1-i, K-1-i)    ← pick function (for partial perms)
```

7.5 CornerDBMaker (Database Builder)

How It Builds the Database

BFS from the SOLVED state, going outward:

```

CornerDBMaker::bfsAndStore():
    cube = solved cube
    Q.push(cube)
    db.setNumMoves(cube, 0)    ← solved state = 0 moves
    curr_depth = 0

    while Q not empty:
        curr_depth++
        if curr_depth == 9: break    ← only store up to depth 8

        for each node in current level:
            for each move m:
                node.apply(m)
                if db.getNumMoves(node) > curr_depth:    ← not yet
visited at this depth
                    db.setNumMoves(node, curr_depth)
                    Q.push(node)
                node.undo(m)

    dbToFile(fileName)

```

This stores the MINIMUM distance from solved for each corner configuration. When solving, looking up the current state gives a lower bound on moves needed.

7.6 HashBitboard

```

struct HashBitboard {
    size_t operator()(const RubiksCubeBitboard &r1) const {
        uint64_t h = r1.bitboard[0];
        for (int i = 1; i < 6; i++) h ^= r1.bitboard[i];
        return (size_t)h;
    }
};

```

Weakness: XOR is commutative — states where faces are permuted might have same XOR.

Better alternative: Zobrist hashing (each (face, position, color) triple gets a random 64-bit value; hash = XOR of all active values). Zobrist is designed exactly for board game state hashing.

PART 8 — ALL INTERVIEW QUESTIONS

SECTION A: Architecture & Design (25 Questions)

A1. Walk me through the design of this project top to bottom.

The project has three independent layers. The **Model layer** defines what a Rubik's Cube is — an abstract base class `RubiksCube` with 18 pure virtual move methods and `isSolved()`. Three concrete classes implement it differently: 3D array (human-readable), 1D array (cache-friendly), and bitboard (fastest). The **Solver layer** is a set of four template classes — BFS, DFS, IDDFS, IDA* — parametrised on the cube type and hash function, so any combination works. The **Pattern Database layer** is a precomputed heuristic: a BFS from the solved state that records the minimum moves to solve every possible corner configuration, stored in ~44 MB using a nibble-packed array.

A2. Why did you use an abstract base class?

To define a contract that any cube representation must satisfy. The 18 move methods are pure virtual — every subclass **MUST** implement them. This lets all solvers work against the interface without knowing the internal storage. It also separates the what (interface) from the how (implementation), which is the Open-Closed Principle — you can add a new representation without touching any solver code.

A3. What is the difference between compile-time and runtime polymorphism? Which did you use and why?

Runtime polymorphism uses virtual functions and vtables — the concrete method is selected at runtime. Overhead: one pointer dereference per call (vtable lookup) plus potential cache miss.

Compile-time polymorphism uses C++ templates — the concrete type is substituted at compile time and calls are direct/inlined. Zero overhead.

I used compile-time polymorphism for solvers (`template<typename T, typename H>`) because `move()` and `isSolved()` are called millions of times in tight search loops. The vtable overhead would compound. The abstract base class uses runtime polymorphism for flexibility, but the solvers never go through the vtable — they hold the concrete type directly.

A4. Why are the solvers templates? Why not just pass `RubiksCube*`?

If solvers held a `RubiksCube*`, every `move()` call would go through a vtable — $18 \times$ thousands of nodes = millions of vtable dereferences per solve. Templates eliminate that: the compiler sees `RubiksCubeBitboard::move()` directly at compile time and can inline it. Also, `unordered_map` needs the concrete type for the hash function — you can't hash through a pointer to abstract base.

A5. What happens if I add a 4th cube representation?

Create a new class inheriting from `RubiksCube`, implement all 18 virtual move methods and `isSolved()`, and define a hash struct. Zero changes to solver code. The templates automatically work with the new type. This is the Open-Closed Principle in practice.

A6. What design patterns are present?

- **Template Method:** `RubiksCube` defines the algorithm skeleton (move dispatch in `move(MOVE)`), subclasses fill in concrete moves.

- **Strategy (via templates):** Solvers are interchangeable strategies, selected at compile time.
 - **Null Object:** Not used, but `randomShuffleCube` could return empty vector as null case.
-

A7. How is memory managed in this project?

Mostly stack and `std::vector` (RAII). The `NibbleArray` owns a `vector<uint8_t>` internally — it's automatically destroyed when out of scope. The pattern database holds a `NibbleArray` member, not a pointer — again RAII. Solvers hold cube copies by value. No raw `new/delete` usage — no memory leaks possible.

A8. Is this code thread-safe?

No, and it doesn't need to be. Each solver instance holds its own copy of the cube, its own `visited` map, and its own `moves` vector. Multiple threads could each create their own solver instance and solve independently — no shared mutable state. The pattern database is read-only during solving (writes only happen during `bfsAndStore`), so multiple readers are safe.

A9. How would you parallelise the solver?

Two approaches: 1. **Parallel IDA***: At the first level of search, assign different starting moves to different threads. Each thread runs IDA* on its subtree and reports the solution. 2. **Bidirectional search**: One thread expands from the scrambled state forward; another thread expands from the solved state backward. They meet in the middle, reducing search depth from d to $d/2$, dropping the work from $O(18^d)$ to $O(18^{(d/2)})$.

A10. Why is the pattern database built by BFS from the solved state, not from the scrambled state?

BFS from the solved state fills the entire database in one pass — every possible corner configuration gets its optimal distance. If you BFS from the scrambled state, you only get distances for states reachable from that particular scramble. The database must be reusable across all scrambles, so it must be built from a universal starting point.

A11. What is the Open-Closed Principle? Does your code follow it?

Open-Closed: software entities should be open for extension but closed for modification. Yes — to add a new solver algorithm or cube representation, you extend (add a new class) without modifying existing classes. The template-based solver design is an example.

A12. How would you add a GUI to this project?

Keep the solver and model layers unchanged. Add a presentation layer that reads the solver's output (`vector<MOVE>`) and animates them. Options: Qt/wxWidgets for a desktop app, OpenGL for a 3D visualization, or compile to WebAssembly for a browser demo. The clean layer separation means the core algorithm needs zero changes.

A13. How did you verify the move implementations are correct?

Apply a move and its inverse: `cube.u(); cube.uPrime()` — should return to solved.
Apply the same move 4 times — should return to solved (quarter turns have order 4).
Apply a move 2 times for `u2` variants — should equal `u2`. Cross-check all 3 representations produce identical states after the same sequence of moves.

A14. Why is the bitboard representation the “fastest”?

Each face fits entirely in one 64-bit CPU register. A face rotation is a single `ror` (rotate right) instruction. Moving stickers between adjacent faces is a handful of bitwise AND/OR/shift operations on 64-bit values — all executing in 1–3 cycles. Compare to the 3D array where a move updates 9–12 individual byte-sized memory locations, each potentially a cache miss.

A15. What is CMake and why use it?

CMake is a cross-platform build system generator. It produces `Makefile` (Linux), `Xcode` project (Mac), or `Visual Studio` solution (Windows) from a single `CMakeLists.txt`. Using it means the project builds identically on any platform without rewriting build files. It also handles include paths — which is how `compat/bits/stdc++.h` gets found automatically.

A16. What is the `compat/bits/stdc++.h` file for?

`bits/stdc++.h` is a GCC extension that includes every standard header at once. Apple Clang (the default Mac compiler) doesn’t ship this header. The `compat/` directory provides a shim that simply `#includes` all the standard headers explicitly, and `CMakeLists.txt` adds this directory to the include path so `#include <bits/stdc++.h>` resolves to the shim.

A17. Why keep three cube representations instead of just one?

For benchmarking and learning. Each representation has different performance characteristics — the project demonstrates the real-world impact of data layout on algorithm speed. Interviewers like seeing you can think about performance tradeoffs, not just correctness.

A18. What would you refactor if you had more time?

1. Split `RubiksCubeBitboard.cpp` into a proper `.h/.cpp` pair.
 2. Add unit tests with Google Test or Catch2.
 3. Replace the priority-queue `IDA*` with a proper recursive `IDA*` for true $O(d)$ space.
 4. Add an edge pattern database for a tighter heuristic.
 5. Add a validity checker to detect physically impossible cube states.
-

A19. Explain the relationship between `PatternDatabase` and `CornerPatternDatabase`.

PatternDatabase is the abstract base: it owns the NibbleArray, handles all storage, file I/O, and getNumMoves/setNumMoves logic. The only pure virtual method is `getDatabaseIndex(cube)` — which converts a cube state to a unique integer index.

CornerPatternDatabase is the concrete subclass. It implements `getDatabaseIndex` using the Lehmer code on corner permutations combined with the base-3 orientation encoding. All the nibble/file machinery is inherited.

A20. What is “God’s Number”? How was it determined?

God’s Number is 20 — the maximum number of moves needed to solve any Rubik’s Cube in the Half Turn Metric. Proved in 2010 by Morley Davidson, John Dethridge, Herbert Kociemba, and Tomas Rokicki using a computer search that checked all 43 quintillion states. They used symmetry reduction to group states into 2.2 billion cosets, then used 35 CPU-years (donated by Google) to verify each coset. The result: no state requires more than 20 moves.

A21. What is the “Half Turn Metric”?

HTM counts each quarter turn AND each 180° turn as a single move. So $F2(180^\circ) = 1$ move, same as $F(90^\circ)$. Alternative is the Quarter Turn Metric (QTM) where $F2 = 2$ moves. God’s Number is 20 in HTM and 26 in QTM.

A22. Why does your hash function XOR all 6 bitboards?

It’s fast (6 XOR operations) and gives a reasonable distribution for typical cube states. The weakness is that XOR is symmetric — two states where faces are swapped could hash identically. For a placement project this is acceptable. Production quality would use Zobrist hashing.

A23. What is Zobrist hashing?

For each (position, color) combination on the cube, pre-generate a random 64-bit number. The hash of a cube state is the XOR of the random values corresponding to every sticker’s current (position, color) pair. This gives much better distribution than a simple XOR of face values because each sticker contributes a different random value at each position.

A24. Why is the DBMaker’s BFS loop: `if curr_depth == 9: break`?

The database only needs to store depths 0–8 because the heuristic is used as a lower bound — any value 0–8 is useful. Depths beyond 8 would require many more BFS levels and much more time to compute. In practice the database is called “cornerDepth8V1” — capturing all corner states solvable in ≤ 8 moves covers the majority of configurations that arise in the search.

A25. What does `reinterpret_cast<const char*>` do in file writing?

The `write()` method expects `const char*`, but our buffer is `const uint8_t*`. Both types are 1-byte integers, but C++ won't let you pass `uint8_t*` where `char*` is expected without a cast. `reinterpret_cast` tells the compiler to treat the same memory address as a different type — safe here because we're just telling the stream “read these bytes” and `char/uint8_t` have identical binary layout.

SECTION B: Algorithm Deep Dive (25 Questions)

B1. What is the difference between BFS and Dijkstra's algorithm?

BFS assumes all edges have equal weight (cost 1 per move), so distance = number of moves. That's correct here — each Rubik's move costs 1. Dijkstra handles variable edge weights with a priority queue. Since all Rubik's moves have the same cost, BFS is the right choice (simpler and faster than Dijkstra for uniform-cost problems).

B2. What is an admissible heuristic?

A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal from n . Formally: $h(n) \leq h^*(n)$ for all n , where $h^*(n)$ is the true minimum cost.

My heuristic is the corner pattern database value: the minimum moves to fix all 8 corners. Since you can't solve the full cube in fewer moves than you need to fix just the corners, this is a valid lower bound.

B3. What is a consistent (monotonic) heuristic?

A consistent heuristic satisfies the triangle inequality: $h(n) \leq \text{cost}(n, n') + h(n')$ — the estimated cost from n is at most the actual cost to a neighbour n' plus the estimated cost from n' . Consistency implies admissibility. The corner DB heuristic is consistent because applying one more move can change the corner DB value by at most 1.

B4. Why is IDA* better than A* for this problem?

A* keeps the entire frontier (open set) in memory. For a Rubik's Cube, the frontier at depth 10 has $\sim 10^{12}$ nodes — impossible to store. IDA* only keeps the current path ($O(d)$ space). The repeated iterations of IDA* cost some time, but with a tight heuristic, most of the search tree is pruned, making the extra time negligible compared to the memory savings.

B5. Can IDA* be suboptimal?

No, if the heuristic is admissible. IDA* is guaranteed to find the optimal (shortest) solution when $h(n)$ never overestimates. If you use an inadmissible heuristic (overestimates sometimes), it might prune optimal paths and return a suboptimal solution.

B6. What is the branching factor of a Rubik's Cube search?

The raw branching factor is 18 (one state for each of the 18 legal moves from any position). In practice, you can reduce it by avoiding redundant moves: - Don't apply the inverse of the just-applied move (e.g., after R, don't apply R' next) - Don't apply moves to the same face consecutively in certain orders This reduces effective branching to ~15. With the heuristic, many branches are pruned further.

B7. What is the effective branching factor with your heuristic?

Hard to measure exactly without profiling, but empirically: solving a 10-move scramble in < 1 second implies the heuristic prunes the search from $O(18^{10}) \approx 3.6$ trillion to a few million nodes. The effective branching factor is roughly 15–16 per level (after removing immediately-reversing moves) and drops further with heuristic pruning.

B8. How does the search know when it found the solution?

`node.isSolved()` checks if `bitboard[i] == solved_side_config[i]` for all 6 faces. The `solved_side_config` is computed once in the constructor — it's the bitboard state of a fresh, unscrambled cube. Comparing 6 64-bit integers is 6 operations.

B9. How do you reconstruct the solution path?

The `move_done` map records, for each cube state reached, which move produced it. Starting from the solved state, repeatedly look up `move_done[current]` to get the move, then invert that move to get the previous state. Collect these moves, then reverse the array to get moves in forward order.

```
solved → invert(move_done[solved]) → prev_state
prev_state → invert(move_done[prev_state]) → prev_prev_state
...until we reach the original scrambled state
```

B10. What is the time complexity of IDA* in the best case?

If the heuristic is perfect (equals the true remaining distance at every node), the search visits only nodes on the optimal path — $O(d)$ nodes where d is the solution depth. In practice, no heuristic is perfect, so the actual complexity is between $O(d)$ and $O(18^d)$.

B11. Why does the DFS solver lack visited-state tracking? Is that a bug?

It's a known limitation. Adding visited-state tracking would require $O(18^d)$ memory — the same as BFS. DFS's advantage IS its $O(d)$ memory usage, which requires sacrificing the visited set. The tradeoff: DFS can revisit states (wasting time) but uses minimal memory. With the hard depth cap (`max_search_depth`), the worst case is bounded.

B12. What is iterative deepening's advantage over BFS for memory?

BFS must store ALL states at the current frontier level in the queue, plus all states in visited. At depth d with branching factor b , that's $O(b^d)$ states. IDDFS only stores the path from root to current node on the call stack — $O(d)$ space. For $b=18$, $d=10$: BFS needs $\sim 18^{10} \approx 3.6$ trillion entries; IDDFS needs just 10.

B13. What is the invert method and why do you need it?

`invert(m)` applies the inverse of move `m`: if `m` is `L`, `invert` applies `L'`; if `m` is `L2`, `invert` applies `L2` (which is its own inverse). It's needed to "undo" a move when backtracking: `node.move(m); ... node.invert(m)` returns `node` to its original state without copying it.

B14. In your BFS, why do you apply and then invert the move on the same node?

Instead of copying the cube for each move, you apply the move, check/enqueue the resulting state, then invert the move to restore the original. This is more memory-efficient — one cube object rather than 18 copies per node. The downside is that `invert` must be correctly implemented and atomic.

B15. What if two different scrambles have the same BFS hash?

The hash would collide — both states map to the same bucket in the `unordered_map`. C++ `unordered_map` handles collisions correctly via chaining. The `==` operator is used to distinguish colliding entries. A hash collision doesn't corrupt the solution — it just adds a bucket chain traversal.

B16. Is your IDA* always finding the OPTIMAL solution?

Yes, because the heuristic is admissible (corner DB is a true lower bound). However, for very deep scrambles (>10 moves), the database only stores values up to 8, so the heuristic weakens — it might say "8 moves minimum" when the true remaining is 15. IDA* still finds the optimal solution but may take longer because less pruning occurs.

B17. What happens if the database file is missing?

`fromFile()` returns false if the file can't be opened. The `IDAStarSolver` constructor calls `cornerDB.fromFile(fileName)` but doesn't check the return value — a bug in the current implementation. The consequence: all heuristic values would return `0xFF` (the initial `NibbleArray` fill), which would be treated as "unreachable" — solver would loop forever or fail an assertion.

B18. How would you implement bidirectional BFS?

Maintain two frontiers: `forward_frontier` starting from the scrambled state, and `backward_frontier` starting from the solved state. At each step, expand the smaller frontier. When a state appears in both frontiers, the path is: forward path to that state + reverse of backward path from that state. Reduces depth from d to $d/2$, cutting work from $O(18^d)$ to $O(2 \times 18^{(d/2)})$.

B19. What is the relationship between DFS and recursion?

DFS naturally maps to recursion — each recursive call represents going deeper down the search tree, and returning is backtracking. The call stack IS the path from root to current node. My `DFSolver::dfs(dep)` is exactly this: `dep` tracks current depth, recursion handles branching, and returning false triggers backtracking.

B20. Could you use Dijkstra's algorithm instead of BFS?

Yes, but it would be unnecessary. Dijkstra handles variable edge costs using a priority queue. All Rubik's moves have cost 1, so the priority queue degenerates to a regular FIFO queue — exactly BFS. Using Dijkstra would just add priority-queue overhead for no benefit.

B21. What is A* and how does it differ from IDA*?

A* maintains an **open set** (priority queue ordered by $f = g+h$) and a **closed set** (visited states). It expands the globally minimum- f node at each step. It finds optimal paths and never re-expands nodes (with consistent heuristic).

IDA* does iterative deepening on the f -bound instead. No open set stored in memory — just the current path. IDA* may re-expand the same node multiple times across iterations but uses $O(d)$ space instead of $O(b^d)$.

B22. What is the “plateau problem” in IDA*?

When the heuristic is very flat (returns the same value for many states), IDA* can't prune effectively and must explore huge regions. This happens when the pattern DB returns 0 for most solved-corner states that still have unsolved edges. Stronger heuristics (combined corner + edge DB) alleviate this.

B23. Explain the concept of “pruning” in search.

Pruning means discarding subtrees we know can't contain the solution. In IDA*, if $g + h > \text{bound}$, we know this path will need more moves than allowed in the current iteration — so we stop expanding it immediately. This can eliminate billions of states from consideration.

B24. What is memoization and did you use it?

Memoization caches the results of function calls. The visited map in BFS/IDA* is a form of memoization — it caches which states have been explored. The pattern database itself is an extreme form of memoization: all 88 million corner heuristic values are precomputed once and stored.

B25. Could you use dynamic programming to solve the Rubik's Cube?

The pattern database construction IS dynamic programming: a BFS that builds up a table of optimal subproblem solutions (optimal moves to solved corners) bottom-up. The full cube can't be solved purely with DP (state space too large to store), but the heuristic leverages a partial DP solution.

SECTION C: Data Structures (20 Questions)

C1. What is a bitboard? Why is it used in games and puzzles?

A bitboard represents a board state as one or more integers, where each bit corresponds to a position or property. Operations on the board become bitwise operations — fast, parallel, and cache-friendly. Used in chess engines (pawn positions, attack patterns), Go engines, and here for Rubik's Cube face states.

C2. What data structure does `unordered_map` use internally?

A hash table with chaining (or open addressing in some implementations). Keys are hashed to a bucket; the key-value pair is stored in that bucket (or its chain). `operator[]` and `find()` are $O(1)$ average, $O(n)$ worst case (all keys hash to same bucket).

C3. How does the queue in BFS work?

`std::queue<T>` is a FIFO container backed by `std::deque`. `push()` adds to the back, `front()` reads the front, `pop()` removes the front. In BFS, each level's states are pushed in order and dequeued in order — guaranteeing level-by-level traversal.

C4. Why does BFS guarantee the shortest path?

Because it explores all states at distance 1 before distance 2, all at distance 2 before distance 3, and so on. The first time the goal is found, it's at the minimum distance — no shorter path exists because all shorter ones have already been explored.

C5. What is the `priority_queue` used in IDA*?

`std::priority_queue<T>` is a max-heap by default. The IDA* implementation uses a custom comparator `compareCube` that makes it a min-heap ordered by $f = g + h$. Nodes with lowest f -cost are expanded first — this is the A* ordering strategy.

C6. What is the time complexity of heap operations?

- `push()`: $O(\log n)$
- `pop()`: $O(\log n)$
- `top()`: $O(1)$

For IDA* with millions of nodes, this log factor matters. A Fibonacci heap would give amortized $O(1)$ decrease-key, but in practice `std::priority_queue` is fast enough due to cache efficiency.

C7. How does `bitset<N>` work in `PermutationIndexer`?

`std::bitset<N>` is a fixed-size sequence of bits. `bitset[i] = 1` sets bit i . `to_ulong()` converts the entire bitset to an unsigned long. In `PermutationIndexer`, `seen.to_ulong() >> (N - perm[i])` extracts the bits to the left of position `perm[i]`

(where left = already-seen digits), and onesCountLookup counts how many of those are set.

C8. What is a hash collision and how is it handled?

A collision is when two different keys produce the same hash value. `unordered_map` handles it by storing both key-value pairs in the same bucket as a linked list (chaining). When looking up a key, the bucket is found via hash, then the chain is searched linearly for the exact key match using `operator==`.

C9. Why store the database as binary bytes rather than text?

Binary is compact — each nibble stores a value 0–8 in 4 bits. Text would need 1–2 characters per value (e.g., “8” = 2 bytes) — 4–8× larger. Binary also reads/writes faster since there’s no parsing. The database is ~44 MB binary; text would be ~200–400 MB.

C10. What is a vector in C++ and how does it differ from an array?

`std::vector<T>` is a dynamic array on the heap. Key differences from a raw array: - **Dynamic size:** can `push_back` without knowing size upfront - **Bounds checking:** `at()` throws `std::out_of_range` - **RAII:** automatically freed when out of scope - **Contiguous memory:** same cache behavior as array

The NibbleArray uses `vector<uint8_t>` so its size is determined at runtime (pattern DB size).

C11. What is a nibble, and where else are nibbles used?

A nibble is 4 bits = half a byte. Used in: - BCD (Binary Coded Decimal): each decimal digit as a nibble - Color depth (4-bit color = 16 colors, early PC graphics) - Hexadecimal: each hex digit is exactly one nibble (0–F = 0–15) - Compression: when values have small range, packing saves memory

C12. What does `arr.at(i)` do vs `arr[i]`?

`at(i)` performs bounds checking and throws `std::out_of_range` if `i >= size`. `arr[i]` does no bounds check — accessing out of bounds is undefined behavior (may crash, corrupt memory, or “work” unpredictably). Use `at()` during development; `operator[]` in performance-critical code after verifying correctness.

C13. What is the `assert()` macro?

`assert(condition)` checks the condition at runtime. If false, it prints a diagnostic and calls `abort()`. Enabled in debug builds, disabled in release builds (`-DNDEBUG`). Used in my code to verify invariants like `pos <= this->size` before nibble operations.

C14. What is the difference between `stack` and `queue`?

- `stack`: LIFO — last in, first out. DFS uses a stack (implicit call stack in recursion, or explicit with iterative DFS).
 - `queue`: FIFO — first in, first out. BFS uses a queue — ensures level-by-level traversal.
-

C15. Why is `std::array` used in `PermutationIndexer` instead of `std::vector`?

`std::array<T, N>` has fixed size known at compile time, stored on the stack — zero heap allocation. `std::vector` allocates on the heap. Since $N=8$ for corners (known at compile time), `array` is faster (no allocation overhead) and the template parameter enforces the size.

C16. What does `reserve()` do in a vector?

Pre-allocates memory for at least the specified number of elements without changing the size. In `NibbleArray::inflate()`, `dest.reserve(this->size)` avoids repeated reallocations as elements are `push_back`-ed — improves performance from $O(n \log n)$ to $O(n)$ total allocation cost.

C17. What is a Lehmer code (factoradic number)?

The Lehmer code represents a permutation in the factorial number system. For permutation `[3,0,7,1,2,5,6,4]`: - At position 0: how many elements to the right are smaller than 3? → 3 (digits 0,1,2) - At position 1: of remaining, how many to right are smaller than 0? → 0 - ...

The Lehmer code `[3,0,5,0,0,1,1,0]` maps to $3 \times 7! + 0 \times 6! + 5 \times 5! + \dots = \text{unique index}$.

C18. What are the possible values in the corner pattern database and what do they mean?

0 = solved corner configuration (0 moves needed)
1–8 = number of moves to solve corners
0xFF = not yet computed (initial fill before BFS runs)

Since the nibble can store 0–15 (4 bits), and move counts are 0–8, this works. Values of 0xF (15 in 4 bits) would appear for entries the BFS didn't reach (depth > 8), and the solver treats them as “need many moves” — still a valid lower bound.

C19. How does the `isFull()` check work in `PatternDatabase`?

`numItems` tracks how many entries have been set (moved from 0xFF to a real value)
`isFull()` returns `numItems == size`

This is incremented in `setNumMoves` only when the old value was 0xF (unvisited):
 if (oldMoves == 0xF): ++numItems

This allows checking if the entire database has been populated without scanning 88 million entries.

C20. What is `fill()` used for in `NibbleArray::reset()`?

`std::fill(begin, end, value)` sets all elements in the range to `value`. In `reset()`, it fills the entire `arr` vector with `val` (default `0xFF`) — resetting all nibbles to “unvisited” state. Equivalent to `memset` but type-safe and works on any iterator range.

SECTION D: C++ Language Questions (20 Questions)

D1. What is `using namespace std` and why is it bad in headers?

`using namespace std` makes all names in the `std` namespace available without the `std::` prefix. In a `.cpp` file, this affects only that translation unit — harmless. In a `.h` file, it affects EVERY file that includes the header, transitively. If two libraries both define `sort`, or if the project’s own code has a function named `distance`, `vector`, or `map`, unexpected name collisions occur. It’s the C++ equivalent of polluting global scope.

D2. What is `uint8_t` and why not use `int`?

`uint8_t` is guaranteed to be exactly 8 bits, unsigned, defined in `<cstdint>`. `int` is typically 32 bits. Using `uint8_t` for values that fit in 0–255: - Saves 4× memory in arrays - Makes intent clear (this value fits in one byte) - Prevents signed/unsigned comparison bugs

D3. What is a pure virtual function?

A virtual function declared with `= 0`. It has no default implementation in the base class — every concrete subclass MUST provide one. A class with any pure virtual function is “abstract” and cannot be instantiated. In my code: `virtual COLOR getColor(FACE face, unsigned row, unsigned col) const = 0;`

D4. What is the vtable?

Every class with virtual functions has a hidden pointer (the “vptr”) added to each object, pointing to the class’s virtual table (vtable). The vtable is an array of function pointers — one per virtual method. When you call a virtual function through a pointer/reference, the runtime looks up the correct function pointer in the vtable and calls it. This enables runtime polymorphism but adds one level of indirection per call.

D5. What is the Rule of Three/Five?

If a class needs a custom destructor, copy constructor, or copy assignment operator, it likely needs all three (Rule of Three). In C++11, add move constructor and move assignment (Rule of Five). `RubiksCubeBitboard` defines `operator=` and `operator==` — it should also ensure correct copy construction (which is provided by default since the class has only POD members).

D6. What is RAII?

Resource Acquisition Is Initialization. Resources (memory, file handles, locks) are acquired in constructors and released in destructors. Since destructors run automatically when objects go out of scope, resources are never leaked. `std::vector`, `std::fstream`, `std::unique_ptr` are all RAII wrappers.

D7. What does `override` do?

The `override` keyword (C++11) tells the compiler that this function is meant to override a virtual function in the base class. If the base class doesn't have a matching virtual function, the compiler gives an error. It prevents silent bugs from typos (e.g., `issolve()` instead of `isSolved()` would silently become a new non-virtual function without `override`).

D8. What is `const` correctness?

Marking methods `const` means they don't modify the object's state. `getColor()` `const`, `isSolved()` `const`, `getNumMoves()` `const` — these can be called on `const` objects and `const` references. Without `const`, calling `getColor()` on a `const RubiksCube&` would be a compile error. It also enables compiler optimizations.

D9. What is `explicit` for constructors?

Prevents implicit conversions. Without `explicit`, a constructor `NibbleArray(size_t size)` would allow `NibbleArray arr = 5` — implicitly constructing from an integer. `explicit` forces you to write `NibbleArray arr(5)` — making the construction intentional.

D10. What is `size_t`?

`size_t` is an unsigned integer type returned by `sizeof` and used for array sizes and indices. Its width matches the platform's pointer size (32-bit on 32-bit systems, 64-bit on 64-bit systems). Using `size_t` for sizes prevents signed/unsigned comparison warnings and correctly handles sizes up to the platform maximum.

D11. What is a template and how does it work?

A template is a blueprint for generating code. `template<typename T, typename H> class BFSolver` generates a new class for each unique (T, H) pair used in the program. The compiler instantiates the template at compile time — `BFSolver<RubiksCubeBitboard, HashBitboard>` is a fully concrete class. Templates have zero runtime overhead because specialization happens at compile time.

D12. What is the ODR (One Definition Rule)?

Each non-inline function/variable must be defined exactly once across all translation units in a program. Violating ODR (e.g., including a `.cpp` file that defines non-inline functions in multiple places) results in linker errors or undefined behavior. Headers should contain

only declarations (not definitions) or use `inline`.

D13. What does `inline` do?

Suggests the compiler replace a function call with the function body directly at the call site — eliminating function-call overhead. Class member functions defined inside the class body are implicitly `inline`. For short frequently-called functions (like bitwise operations), inlining can significantly improve performance.

D14. What is `static_cast` vs `reinterpret_cast`?

- `static_cast<T>`: safe, checked at compile time. Used for related types (int to float, derived to base pointer, enum to int).
 - `reinterpret_cast<T>`: tells the compiler to treat the same bits as a different type. Unsafe — bypasses type system. Used in `PatternDatabase` to cast `uint8_t*` to `char*` for file I/O.
-

D15. What is `enum class` and why use it over `enum`?

`enum class` (scoped enum, C++11) keeps enumerators within the enum's scope — `FACE::UP` instead of just `UP`. It prevents name collisions (two enums can have values named `UP`) and prevents implicit conversion to integers (you need an explicit cast). My code uses `enum class FACE`, `enum class COLOR`, `enum class MOVE`.

D16. What is `auto` and when is it useful?

`auto` deduces the type from the initializer at compile time. Zero runtime overhead. Useful for long type names (`auto it = map.begin()` vs `unordered_map<T, bool, H>::iterator it = ...`) and for range-based for loops (`for (auto m : moves)`). In my code: `auto curr_move = RubiksCube::MOVE(i)`.

D17. What is a `struct` vs a `class` in C++?

The only difference is default access: `struct` members are public by default, `class` members are private. `HashBitboard` is a `struct` because it's a simple functor with one public operator(). The `Node` `struct` in IDA* is also public-by-default since it's a private inner type used only within the class.

D18. What is `std::chrono` and how did you use it?

`std::chrono` provides a type-safe time library. `high_resolution_clock::now()` returns a `time_point`. Subtracting two `time_points` gives a `duration`. `duration<double>` converts to seconds as a `double`. Used in `main.cpp` to measure solver elapsed time:

```
auto t0 = chrono::high_resolution_clock::now();
// ... solve ...
auto t1 = chrono::high_resolution_clock::now();
double secs = chrono::duration<double>(t1 - t0).count();
```

D19. What is `srand(time(0))` used for?

Seeds the pseudo-random number generator with the current time. Without seeding, `rand()` returns the same sequence every run. `time(0)` returns seconds since epoch — changes every second, giving different shuffles each run. Better alternatives: `<random>` header with `mt19937` (Mersenne Twister).

D20. What is `pair<A,B>` and `make_pair()`?

`std::pair<A,B>` bundles two values together. `make_pair(a, b)` constructs a pair (type-deduced). Used in IDA* to return both the solved cube and the next bound from `IDAstar()`: `return make_pair(node.cube, bound)`. Access members with `.first` and `.second`.

SECTION E: OOP & Design Principles (10 Questions)

E1. What are the SOLID principles? Which ones does your code follow?

- **Single Responsibility:** Each class has one job — `NibbleArray` handles nibble storage, `PatternDatabase` handles DB logic, `CornerPatternDatabase` handles corner-specific indexing. ✓
 - **Open/Closed:** Add new cube representations or solvers without modifying existing code. ✓
 - **Liskov Substitution:** Any `RubiksCube&` can be replaced by `RubiksCubeBitboard&` or `RubiksCube3dArray&` without breaking correctness. ✓
 - **Interface Segregation:** `RubiksCube` could be split (printer interface, mover interface) — not fully applied.
 - **Dependency Inversion:** Solvers depend on the abstract `RubiksCube` interface, not concrete types. ✓ (via templates)
-

E2. What is inheritance vs composition? Did you use both?

- **Inheritance:** “is-a” relationship. `RubiksCubeBitboard` IS-A `RubiksCube`. Used for the cube hierarchy and database hierarchy.
- **Composition:** “has-a” relationship. `PatternDatabase` HAS-A `NibbleArray`. `CornerPatternDatabase` HAS-A `PermutationIndexer`. Used for data structure embedding.

Both are used. The rule of thumb: prefer composition over inheritance unless the IS-A relationship is truly semantic.

E3. What is an abstract class?

A class with at least one pure virtual method. It can't be instantiated directly but can be used as a type for pointers/references. `RubiksCube` and `PatternDatabase` are both abstract — they define interfaces for their concrete subclasses.

E4. What is the difference between method overriding and method overloading?

- **Overriding:** A subclass provides a new implementation for a virtual method inherited from the base class. `RubiksCubeBitboard::u()` overrides `RubiksCube::u()`.
 - **Overloading:** Multiple methods with the same name but different parameter signatures in the same class. `PatternDatabase::setNumMoves(const RubiksCube&, uint8_t)` and `setNumMoves(uint32_t, uint8_t)` are overloads.
-

E5. Why did you choose to make `getColorLetter` and `getMove` static?

These functions don't depend on any instance state — they're pure conversions (`COLOR` → `char`, `MOVE` → `string`). Making them `static` signals this: they belong to the class conceptually but need no object to call. Call them as `RubiksCube::getColorLetter(c)` without creating a cube.

E6. What is encapsulation and where is it used here?

Encapsulation = hiding internal implementation details behind a public interface. `NibbleArray` hides its `arr` vector and bit-manipulation logic; users just call `get(pos)` and `set(pos, val)`. `PatternDatabase` hides the `NibbleArray` and exposes only `getNumMoves/setNumMoves`. Users of the DB never see nibble operations.

E7. Could you make `RubiksCube` a pure interface (no code)?

Yes — move `print()`, `move()`, `invert()`, `randomShuffleCube()` to a separate utility class or make them non-virtual template functions. But then you lose the convenience of calling `cube.print()` polymorphically. The current design is a compromise: the base class provides default implementations for non-performance-critical methods.

E8. What is a functor? How is it used here?

A functor is a class with `operator()` defined — makes instances callable like functions. `HashBitboard`, `compareCube` in `IDA*` are functors. They're passed as template parameters to `unordered_map<T, bool, HashBitboard>` and `priority_queue<..., compareCube>`, where the container calls `operator()` to hash/compare elements.

E9. What is the difference between deep copy and shallow copy?

- **Shallow copy:** Copy the pointer/reference, not the pointed-to data. Both copies point to the same memory.
- **Deep copy:** Copy the actual data — both objects are independent.

`RubiksCubeBitboard::operator=` performs a deep copy — it copies all 6 `uint64_t` bitboard values. The `NibbleArray` copy (default) copies the entire `vector<uint8_t>` — also deep. No shared pointers means no accidental shallow-copy bugs.

E10. Why does `PatternDatabase` have a private default constructor?

```
PatternDatabase(); // private!
```

This prevents subclasses or external code from constructing a `PatternDatabase` without providing a size. There's no meaningful "default size" for a pattern database — forcing the size argument ensures the `NibbleArray` is correctly allocated. It's a form of invariant enforcement.

SECTION F: Performance & Optimization (10 Questions)

F1. What is cache locality and why does the bitboard help?

Modern CPUs cache recently accessed memory in L1/L2/L3 caches. Accessing memory far from what you just accessed ("cache miss") takes 100–300 cycles instead of 1–4. The bitboard stores all 8 edge/corner positions of a face in one 64-bit word — one cache line. The 3D array stores each cell as a separate `uint8_t`, and a move might touch cells scattered across the 54-byte array. The bitboard has better spatial locality.

F2. How much memory does each cube representation use per state?

Representation	Memory per state
3D Array	$6 \times 9 \times 1 \text{ byte} = 54 \text{ bytes}$
1D Array	54 bytes
Bitboard	$6 \times 8 \text{ bytes} = 48 \text{ bytes}$

At 10 million states in BFS: 3D array uses 540 MB, bitboard uses 480 MB — the difference matters at scale.

F3. What is loop unrolling and does your code benefit from it?

Loop unrolling replaces a loop with repeated copies of the loop body to reduce branch/counter overhead. The compiler can auto-unroll short loops. The `for (int i = 0; i < 6; i++)` in `isSolved()` and `HashBitboard::operator()` are candidates — the compiler likely unrolls these since the bound is small and fixed.

F4. What is branch prediction and how does it affect your DFS?

Modern CPUs predict whether a branch (if/loop) will be taken or not, and speculatively execute ahead. Mispredictions cost ~15 cycles. In DFS, the `if (dep > max_search_depth) return false` branch is almost always NOT taken (most nodes are not at the depth limit), so the predictor learns this quickly. The `if (isSolved())` branch is rarely taken — also predictable.

F5. How would you profile this code?

Use `gprof` (GCC profiler), `perf` (Linux), or Instruments (Mac) to identify hotspots. Likely hotspots: `getNumMoves()` (called millions of times), `getDatabaseIndex()` (called for every node), `getColor()` (called 8 times per corner per node). Once

identified, optimise the innermost operations first.

F6. What is the time spent building the corner database?

BFS from solved state, exploring all states reachable in ≤ 8 moves. ~ 88 million states $\times$ 18 moves each = ~ 1.6 billion `move()` and `invert()` operations. On modern hardware with bitboard representation: roughly 30–60 seconds. The result is saved to disk so it's built once and reused for all future runs.

F7. Could the database be made smaller?

Yes — exploit symmetry. The Rubik's Cube has 48 symmetries (rotations and reflections of the cube). States related by symmetry have the same minimum solution depth. Grouping them reduces the database by 48 $\times$, from 88M to ~ 1.8 M entries — fits in ~ 1 MB. This requires transforming any input state to its canonical representative before lookup.

F8. How does memory usage affect IDA*?

IDA* stores only the current path plus the `visited` and `move_done` maps (which can grow large with the PQ-based implementation). The path itself is just a `vector<MOVE>` of length `d` — negligible. The `visited` map can grow to $O(\text{explored nodes})$ — the main memory concern. Pure recursive IDA* eliminates even this (no `visited` map, relies on the `f`-bound to prevent cycling).

F9. What is the bottleneck of your current IDA* implementation?

The priority queue operations: `push()` and `pop()` are $O(\log n)$. With millions of nodes, this $\log$ factor adds up. Also, the `unordered_map` lookups for `visited` and `move_done` have overhead from hashing and potential collisions. A pure recursive IDA* with no map would be faster but requires careful handling to avoid infinite loops.

F10. What is $O(n \log n)$ vs $O(n)$ and when does it matter?

- $O(n \log n)$: e.g., heap operations, sorting. For $n=10^9$, $n \log n \approx 30 \times 10^9$ operations.
- $O(n)$: linear scan. For $n=10^9$, 10^9 operations.

For the database BFS ($n=88$ million), the difference between $O(n)$ nibble-array operations and $O(n \log n)$ priority-queue operations is significant — the nibble operations are $\sim 3\times$ faster.

SECTION G: Testing (8 Questions)

G1. How would you write unit tests for the move operations?

```
// Test: applying a move and its inverse returns to start
cube = solved
```

```

cube.u(); cube.uPrime();
assert(cube.isSolved())    // for all 18 moves

// Test: applying a quarter move 4 times returns to start
cube = solved
cube.u(); cube.u(); cube.u(); cube.u();
assert(cube.isSolved())    // for L,R,U,D,F,B

// Test: applying same move to all 3 representations gives same result
cube3d = solved; cube1d = solved; cubeBB = solved;
cube3d.u(); cube1d.u(); cubeBB.u();
assert(cube3d == cube1d == cubeBB)    // state should be identical

```

G2. How would you test the corner database?

```

// Test: solved state has DB value 0
db = loaded_database
assert(db.getNumMoves(solved_cube) == 0)

// Test: one-move scrambles have DB value 1
for each move m:
    cube = solved; cube.apply(m)
    assert(db.getNumMoves(cube) == 1)    // since corner moves by 1

// Test: DB is a lower bound (never overestimates)
for many random scrambles:
    assert(db.getNumMoves(cube) <= actual_solve_depth)

```

G3. What is regression testing?

Testing that existing functionality still works after new code is added. After adding a new cube representation, re-run all move-correctness tests to verify no existing representations broke. Automated test suites (Google Test, Catch2) make this easy — run them after every change.

G4. How do you test the solver end-to-end?

```

for i in range(1000):
    cube = solved
    shuffle_moves = cube.randomShuffle(random_depth)
    solution = solver.solve(cube)

    // Apply solution to original scramble
    reconstructed = apply_moves(original_cube, solution)
    assert(reconstructed.isSolved())

    // Optional: check optimality
    assert(len(solution) == len(shuffle_moves))    // only for short
scrambles

```

G5. What is the difference between unit tests and integration tests?

- **Unit tests:** Test one component in isolation (e.g., `NibbleArray::get/set`, one cube move). Fast, focused.

- **Integration tests:** Test multiple components working together (e.g., build DB, load DB, solve cube end-to-end). Slower, tests interactions.

Both are needed — unit tests catch individual bugs, integration tests catch interface mismatches.

G6. What edge cases would you test?

- Already-solved cube (solution = empty, 0 moves)
 - One-move scramble (trivially solved)
 - Maximum-depth scramble (stress test)
 - Same scramble run twice (determinism with same seed)
 - Corrupted database file (graceful error handling)
 - Invalid cube state (detect and reject)
-

G7. What would you use for a C++ testing framework?

Google Test (gtest): Industry standard, great assertion macros (EXPECT_EQ, ASSERT_TRUE), parameterised tests, fixtures.

Catch2: Header-only, easy setup, BDD-style tests.

doctest: Extremely lightweight header-only.

For this project I'd use Google Test since it integrates cleanly with CMake via FetchContent.

G8. How would you test the NibbleArray in isolation?

```
NibbleArray arr(10);
```

```
// Test get default value
for i in 0..9: assert(arr.get(i) == 0xF)
```

```
// Test set and get round-trip
arr.set(0, 3); assert(arr.get(0) == 3)
arr.set(1, 7); assert(arr.get(1) == 7)
arr.set(9, 0); assert(arr.get(9) == 0)
```

```
// Test adjacent writes don't corrupt each other
arr.set(0, 5); arr.set(1, 2);
assert(arr.get(0) == 5) // first unaffected
assert(arr.get(1) == 2) // second correct
```

```
// Test values > 4 bits are masked
arr.set(0, 0xFF);
assert(arr.get(0) == 0x0F) // only lower 4 bits stored
```

SECTION H: HR & Behavioral (20 Questions)

H1. Tell me about yourself and this project.

Script: “I’m Shoaib Samim, a [year] student at [college] studying [branch]. I built this Rubik’s Cube solver in C++ as a deep dive into AI search algorithms and low-level performance optimization. The project implements four different solving algorithms — from naive BFS to IDA* — and three internal representations of the cube. The most challenging part was building the corner pattern database: a 44 MB precomputed heuristic that makes IDA* fast enough to solve arbitrary scrambles in under a second. The project taught me a lot about the real cost of data structure choices — the bitboard representation vs the 3D array is a perfect example of how the same algorithm runs at different speeds depending on how you lay out memory.”

H2. Why did you choose this project for placements?

“I wanted a project that demonstrates algorithmic depth, not just framework knowledge. Any placement candidate can build a CRUD app. This project requires understanding heuristic search, memory-efficient data structures, C++ performance optimization, and abstract design — skills that transfer directly to systems engineering, backend optimization, and competitive programming. It also has an engaging visual element that makes it easy to demo.”

H3. What was the most difficult part?

“The bitboard move logic. Each Rubik’s Cube move affects one face plus 3 cells on each of 4 adjacent faces. Getting the exact bit positions right — which 3 slots of the adjacent face change, in what order — required drawing diagrams and building small verification programs. One off-by-one error in bit shifting and the move looks correct for simple cases but fails for complex ones. I also had to ensure that `move()` followed by `invert()` always returns the exact original bitboard state.”

H4. What would you add if you had more time?

“Three things: First, a proper recursive IDA* to replace the PQ-based approach — it would have true $O(\text{depth})$ space. Second, an edge pattern database alongside the corner DB, taking the maximum of both as the heuristic — significantly tighter lower bound. Third, automated tests using Google Test to verify move correctness, database integrity, and solver optimality.”

H5. What did you learn from this project?

“Several things. One: the gap between theoretical complexity and practical performance. BFS is $O(18^d)$ and so is IDA*, but IDA* + heuristic solves problems BFS can’t touch because the heuristic makes the constant factor exponentially smaller. Two: data layout matters as much as algorithm choice — switching from 3D array to bitboard gave a 3–5× speedup for the same algorithm. Three: abstractions have costs — virtual dispatch through a base class pointer vs template instantiation is a real performance difference in tight loops.”

H6. How would you explain this project to a non-technical person?

“A Rubik’s Cube has more possible states than there are grains of sand on Earth. My program finds the sequence of moves to solve it. Instead of trying every possible combination (which would take longer than the age of the universe), it uses a smart strategy: it precomputes a shortcut table that says ‘from any arrangement of the 8 corner pieces, you need at least X moves to solve.’ Using this hint, it skips most dead-end paths and finds the solution in under a second.”

H7. Describe a bug you encountered and how you fixed it.

“The bitboard face rotation was off by 16 bits. The rotateFace function shifts the bitboard left by 16 bits (rotating 2 positions, since each position is 8 bits). I initially implemented it as `side >> 48` to get the wraparound bits, but the mask was wrong — I got the right number of bits but from the wrong positions. I fixed it by carefully drawing the 8-position layout on paper, then writing a test that applied the rotation 4 times to a solved face and verified it returned to the original state.”

H8. How do you handle a problem you don’t know the solution to?

“I break it down. For the Lehmer code, I’d never implemented it before. I read the theory (Wikipedia + a few papers), understood the $O(N^2)$ naive approach, then understood WHY the bitset trick makes it $O(N)$. I implemented the naive version first to verify correctness, then replaced it with the $O(N)$ version and checked the outputs matched. Concrete, verifiable steps — not guessing.”

H9. Have you worked in a team? What’s your role?

Answer based on your actual experience. If you’ve done team projects: describe your specific contribution, how you handled conflicts, how you reviewed each other’s code. If this was solo: “This was an individual project. I designed and implemented everything myself — from the abstract data model to the heuristic database. In a team I would probably take the algorithmic/optimization role and enjoy reviewing others’ code for correctness and performance.”

H10. Where do you see yourself in 5 years?

“Working on systems or infrastructure problems where both algorithm design and performance matter — distributed systems, databases, or ML infrastructure. This project is my way of proving I can go deeper than surface-level API usage. In 5 years I want to be the person a team calls when something is too slow or too complex to brute-force.”

H11. What’s a weakness in your project you’d fix today?

“The missing error handling for the database file. If `cornerDB.fromFile(fileName)` fails, the solver silently runs with all heuristic values returning garbage (0xFF), causing it to loop forever. A one-line fix: check the return value and throw an exception with a clear message. I’d fix this immediately.”

H12. Why C++ and not Python or Java?

“The core bottleneck is speed — BFS, IDA*, and pattern database lookups happen millions of times per second. Python would be 50–100× slower without C extensions, making the 44 MB database approach impractical. Java would be closer but still 2–5× slower and harder to control memory layout (JVM overhead, GC pauses). C++ gives direct control over memory layout (the bitboard layout is explicitly designed for cache efficiency) and zero-overhead abstractions through templates.”

H13. How do you ensure code quality?

“Code review mentally before committing — can I explain every line? Are the variable names clear? Is there a simpler way? For this project: write test cases for each move, verify all three representations produce identical results for the same sequence of moves, verify the solver returns to solved state after applying the solution. In a team: pair review, automated CI.”

H14. Tell me about a time you optimized something.

“The hash function. My initial hash for HashBitboard was summing all 6 uint64_t values. I noticed unusually high collision rates because many cube states differed only in face order, which produced the same sum. Switching to XOR (not sum) improved distribution significantly. If I were building production code I’d switch to Zobrist hashing for even better properties.”

H15. How do you learn new technologies?

“By building something real. For this project: I read about A* and IDA* from textbooks and papers, then implemented both and compared outputs. Understanding came from debugging — when my IDA* wasn’t finding the right solution, I traced through the f-value calculations step by step until I found the issue. Reading alone doesn’t stick; implementation forces real understanding.”

H16. What is your biggest technical achievement?

“Building the corner pattern database. It’s not complex conceptually — BFS from solved state — but getting it right required: understanding the mathematical encoding of corner permutations and orientations, implementing the Lehmer code for O(N) ranking, packing the result into nibbles to halve memory, and verifying the loaded database gives correct heuristic values. Each component was independently testable, which made debugging manageable. The result: a 44 MB file that makes an otherwise-intractable search run in milliseconds.”

H17. How do you handle conflicting design opinions?

“With data. If someone says ‘3D array is cleaner than bitboard,’ I agree it’s more readable — and show the benchmark. Both opinions can be right in different contexts. I implemented all three representations precisely to have this conversation grounded in evidence rather than preference.”

H18. Are you comfortable reading other people’s code?

“Yes. The PermutationIndexer came from a well-known algorithm (Lehmer code) but with an optimization (bitset + precomputed lookup) I hadn’t seen before. I read the code, understood why the bitset trick gives $O(N)$ instead of $O(N^2)$, and can now explain it — that’s what reading code means to me, not just knowing what it does, but knowing why.”

H19. Why should we hire you?

“Because I go deep. Most candidates can use libraries — I understand what’s inside them. This project required implementing a hash map (conceptually), a memory-packed array, a factorial number system encoder, and four search algorithms from first principles in C++. The same depth I applied here is what I’ll apply to whatever problem you give me.”

H20. Do you have any questions for us?

Always ask: - “What does the tech stack look like for the team I’d join?” - “What does a typical first 3 months look like for a new hire?” - “What’s the biggest technical challenge the team is working on right now?” - “How is code quality maintained — code review process, CI/CD?”

SECTION I: Curveball Questions (10 Questions)

I1. What if I told you 10 moves is trivial and I want you to solve a 20-move scramble?

“The solver handles arbitrary scrambles — depth 20 is within IDA*’s capability. It would be slower than depth 10 (exponentially more states to search) because the corner database heuristic weakens for deep scrambles — it can say at most ‘8 moves remain’ even when 15 are needed. A stronger heuristic (corner DB + edge DB, taking max) would improve this. With only the corner DB, depth-20 might take several seconds to minutes depending on the specific scramble.”

I2. Is your heuristic perfect? What would a perfect heuristic look like?

“No. A perfect heuristic for each state would equal the actual minimum moves needed. Building it would require precomputing ALL 43 quintillion cube states — the full database would be ~43 exabytes. In practice, Korf’s algorithm uses 3 disjoint pattern databases whose values can be summed (additive, not max) for a tight heuristic that solves any scramble in seconds.”

I3. What happens if the cube is physically disassembled and reassembled incorrectly?

“The resulting state might be outside the reachable state space — violating corner parity, edge parity, or total parity. The solver would loop forever since no sequence of legal moves can reach the solved state from an impossible configuration. A validity checker should verify these three parity conditions before attempting to solve.”

I4. Can you solve the cube in the minimum number of moves?

“For shallow scrambles (≤ 10 moves), yes — IDA* with an admissible heuristic finds the optimal solution. For deeper scrambles, the heuristic weakens and IDA* still finds a solution but not necessarily the shortest one. To always find the minimum: need a tighter heuristic (Korf’s 3-database approach) or precompute the full state space.”

I5. How would you scale this to a 4x4 Rubik’s Cube (Rubik’s Revenge)?

“The 4x4 cube has different mechanics — ‘parity errors’ can occur that don’t exist in 3x3. The number of states is vastly larger ($\sim 7.4 \times 10^{45}$). The bitboard would need to be redesigned (can’t fit a 4x4 face in 64 bits easily — would need wider integers or multiple words per face). The pattern database approach still applies but requires different corner/edge groups. IDA* would still be the solving algorithm of choice.”

I6. What if two corner states have the same database index? Is that possible?

“No — the index is constructed to be a bijection (one-to-one mapping). The Lehmer code gives a unique integer for each of the $8!$ permutations, and the base-3 orientation encoding gives a unique integer for each of the 3^7 orientation combinations. Multiplying: $\text{rank} \times 2187 + \text{orientationNum}$ is unique for each (permutation, orientation) pair.”

I7. Your hash function XORs all faces — what if two solved cubes with different scrambles give the same hash?

“That would be a collision — two different states hashing to the same value. The `unordered_map` handles this correctly with chaining: it stores both states in the same bucket and uses `operator==` to distinguish them. A collision doesn’t corrupt the result — it just means a slightly slower lookup. For correctness, the key is that `operator==` is correct (it compares all 6 bitboards).”

I8. Why 8 bits per position in the bitboard and not fewer?

“A color needs 6 possible values (6 faces = 6 colors). The minimum bits to represent 6 values is 3 bits ($2^3=8 \geq 6$). Using 3 bits per position would complicate extraction — position boundaries wouldn’t align with byte boundaries, making masking harder. 8 bits (one byte per position) keeps operations simple and clean at the cost of some unused bits. The simplicity is worth the space.”

I9. The `getCorners()` function in the bitboard returns a `uint64_t`. Could it overflow?

“Yes — I need to be careful. The function shifts `ret` left by 5 bits for each of 8 corners: total shift = 40 bits. Since `ret` is `uint64_t` (64 bits) and each corner contributes 5 bits, the maximum value after 8 corners is $8 \times 5 = 40$ bits — well within 64 bits. No overflow.”

I10. What if someone calls `solve()` on an already-solved cube?

“In BFS: the loop immediately dequeues the initial node, finds it solved, and returns an empty path. In DFS: `dfs(1)` immediately returns true with no moves. In IDA*: `h(solved_cube) = 0`, so the initial bound is 0, the search immediately finds the solved state and returns an empty solution vector. All solvers handle this correctly — the demo’s `printMoves` function even handles the empty case gracefully.”

PART 9 — COMPARISON TABLES

Solver Comparison

Property	BFS	DFS	IDDFS	IDA*
Time complexity	$O(18^d)$	$O(18^d)$	$O(18^d)$	$O(18^d)$ / pruning)
Space complexity	$O(18^d)$	$O(d)$	$O(d)$	$O(d)$ [recursive]
Finds optimal?	Yes	No	Yes	Yes (admissible h)
Uses heuristic?	No	No	No	Yes
Practical for 20 moves?	No	No	No	Yes (with good h)
Max practical depth	~6	~8	~8	~20
Memory needed at $d=10$	~TBs	bytes	bytes	bytes (pruned)

Cube Representation Comparison

Property	3D Array	1D Array	Bitboard
Memory per state	54 bytes	54 bytes	48 bytes
Readability	High	Medium	Low
Move speed	Slow	Medium	Fast
Cache friendliness	Poor	Good	Excellent
Hash computation	Moderate	Moderate	Fast (XOR)
Debug-ability	Easy	Moderate	Hard

Data Structure Comparison

Structure	Purpose	Key Operation	Complexity
NibbleArray	Compact storage	get/set nibble	$O(1)$
PatternDatabase	DB wrapper	getNumMoves	$O(1)$
PermutationIndexer	Rank permutation	rank()	$O(N)$
unordered_map	Visited states	lookup	$O(1)$ avg
priority_queue	IDA* ordering	push/pop	$O(\log n)$
queue	BFS frontier	push/pop	$O(1)$

PART 10 — PHRASES TO USE IN THE INTERVIEW

Use these naturally — they show depth:

- *“This is a lower bound because...”* (when discussing heuristic)
- *“The effective branching factor is reduced by...”* (when discussing pruning)
- *“I chose compile-time polymorphism here because vtable overhead compounds in tight loops”*
- *“The Lehmer code gives a bijection between permutations and integers in $O(N)$ time using a bitset trick”*
- *“Nibble packing halves memory at the cost of two bitwise operations per access — worth it at 88 million entries”*
- *“God’s Number — 20 — means any valid scramble can be solved in at most 20 moves”*
- *“The database is built once by BFS from the solved state and reused across all solving sessions”*
- *“All three representations satisfy the same abstract interface — swapping them requires changing one line in main”*

End of Document — Shoaib Samim — Rubik’s Cube Solver Interview Prep

PART 11 — CAMERA SCANNER

Pipeline Overview

Physical Cube → Webcam (OpenCV VideoCapture)
→ Per-face capture with hue-bucket auto-classification
→ Interactive UI: click any wrong sticker to cycle its colour
→ 6 faces confirmed → 54-colour array
→ Final review window (click-to-edit all faces)
→ buildCubeFromScan() → RubiksCubeBitboard
→ IDA* / IDAstarSolverMT → Solution moves

How to Run

```
# Scan physical cube + multithreaded solve (recommended for demo)
./rubiks_cube_solver --scan --fast ../Databases/cornerDepth8V1.txt

# Scan + single-threaded solve
./rubiks_cube_solver --scan ../Databases/cornerDepth8V1.txt
```

Scanning UI (What the User Sees)

Split-screen window — camera feed on the left, cube-net panel on the right.

The cube net is drawn in the standard cross layout:

```
      [UP]
[L] [FRONT] [R] [B]
      [DN]
```

For each of the 6 faces (in order U, F, R, L, B, D):

1. **Live view** — camera shows the cube with a yellow 3×3 grid overlay. The current face is highlighted with a yellow border on the net panel.
2. **User presses SPACE** — the frame freezes, the 9 stickers are sampled and colour-classified with hue-buckets, and the classified colours appear on that face in the net panel.
3. **Manual correction** — user can **click any wrong sticker** on the current face; each click cycles the colour (W→G→R→B→O→Y→W). Centres are locked (they define the face's colour).
4. **SPACE again** → confirmed. Move on to next face. **R** → redo (recapture).
5. After all 6 faces → **Final Review** window opens with the full cube net. User can click any sticker on any face for last-mile corrections.
6. **SPACE / ENTER** → sends the 54-colour array to `buildCubeFromScan()` → solver runs.

Hue-Bucket Classification (The Auto-Guess)

Kept simple deliberately — the user has manual override, so the classifier only needs to guess “close enough”:

```
COLOR classifyByHue(Vec3b bgr) {
    Vec3b hsv = bgrToHSV(bgr);
    int H = hsv[0], S = hsv[1], V = hsv[2];
    // Two-tier white check (handles both bright-tinted and dim
    // whites)
    if (V > 180 && S < 100) return WHITE;
    if (S < 60) return WHITE;
    // Hue buckets for saturated colours
    if (H < 8 || H > 170) return RED;
    if (H < 16) return ORANGE; // narrow orange
    if (H < 42) return YELLOW; // wide yellow
    if (H < 90) return GREEN;
    return BLUE;
}
```

Why simple hue-buckets instead of Lab-distance auto-calibration?

I tried both. Auto-calibration (learn face-centre BGRs during scan, classify all 54 stickers via nearest Lab distance) was more accurate in ideal lighting but had a nasty UX bug: it silently overwrote the user's manual corrections. The hybrid “auto-cal only for un-clicked stickers” also caused surprises — stickers would change after the user thought they'd confirmed them. So I stripped auto-cal out entirely and trusted the manual UI. Predictability beats accuracy when the user is a human in the loop.

HSV vs BGR for Colour Detection

Why HSV?

RGB/BGR mixes colour with brightness. A dark red and a bright red have very different RGB values but similar HSV hue. HSV separates: - **H** (Hue): the actual colour (0-180 in OpenCV — 0-360° halved to fit in uint8_t) - **S** (Saturation): how vivid (0=grey, 255=pure colour) - **V** (Value): brightness

Hue stays stable across lighting changes, so hue-thresholding is much more robust than RGB thresholding.

Sticker Sampling

`samplePatch(frame, roi, row, col)` grabs a 16×16 pixel patch centred on the cell (not the whole cell — sticker edges have shadows/gaps from adjacent stickers). Uses `cv::mean` to average, returns one `Vec3b` BGR value.

Colour Palette for the Cube Net

`colorToBGR(COLOR)` maps the internal `COLOR` enum to BGR values for on-screen drawing: | Colour | BGR | Notes | — — — — — | | WHITE | (255,255,255) | pure white | | GREEN | (0,200,50) | slightly darker for contrast | | RED | (0,0,220) | pure red | | BLUE | (220,90,0) | slightly desaturated for contrast on dark bg | | ORANGE | (0,140,255) | distinct from yellow | | YELLOW | (0,230,230) | distinct from orange |

Mapping Scanned Colours to the Bitboard

The scanner emits `faceColors[54]` in **face-major, row-major** order: - `faceColors[f*9 + r*3 + c]` = colour of face `f`, row `r`, col `c` - Face order: 0=UP, 1=LEFT, 2=FRONT, 3=RIGHT, 4=BACK, 5=DOWN

The bitboard stores each face as `uint64_t` where positions 0-7 are around the face (clockwise from top-left), position 8 is the centre (implicit — never stored, always returned as the face's own colour):

```
static const int posMap[3][3] = {{0,1,2},{7,8,3},{6,5,4}};
// (0,0)→0 (0,1)→1 (0,2)→2
// (1,0)→7 (1,1)→8 (1,2)→3
// (2,0)→6 (2,1)→5 (2,2)→4
```

`buildCubeFromScan()` iterates 6 faces × 9 stickers, uses `posMap` to find the bitboard slot, and OR's `(1 << color) << (8 * pos)` into `bitboard[face]`.

Validation

`validateScan()` runs before solving: counts each colour, verifies each appears exactly 9 times. Rejects impossible cube states early (e.g. 10 whites and 8 reds means at least one sticker is misclassified).

Interview Questions About the Scanner

“Why keep manual correction if auto-classification exists?”

Vision-based colour detection is inherently fragile — lighting, camera white-balance, cube sticker quality all vary. No pure-vision approach hits 100%. A hybrid UI where the algorithm makes a fast guess and the user corrects any misses is more reliable and *takes seconds*. It’s the same design mobile Rubik’s cube apps use.

“What is OpenCV?”

Open Source Computer Vision Library — the industry-standard image processing library for C++/Python. I use: VideoCapture (webcam), cvtColor (BGR↔HSV colour space conversion), cv::mean (patch averaging), drawing primitives (rectangle, putText, line), imshow/waitKey (windowing), and setMouseCallback (for the click-to-fix editor).

“How does the click-to-fix editor work?”

setMouseCallback registers a C-style callback with OpenCV that fires on mouse events in the window. I compute the cube-net layout at fixed pixel dimensions (no runtime resize), so click (x, y) maps deterministically to a (face, sticker) index. Each click cycles the colour enum ((int)color + 1) % 6 and marks that sticker as manually edited.

“How do you handle the coordinate mapping when the camera and cube-net are side-by-side?”

The combined view is a horizontal hconcat of [resized camera | cube-net panel]. In the mouse callback, I check $x - \text{camWidth}$ — if negative, the click was on the camera side (ignored); if positive, that’s the offset into the panel, and I map it to (face, sticker) using the panel’s layout constants.

“How does the scanner map colours back to the solver’s data structure?”

The faceColors[54] array from the scanner is in face-major, row-major order. buildCubeFromScan() uses a posMap[3][3] lookup to convert (row, col) into the bitboard’s position index (0-7 clockwise from top-left, centre implicit), then OR’s the colour bit into the correct 8-bit slot of bitboard[face].

“What happens if the scan is invalid?”

validateScan() counts each colour. If any colour isn’t exactly 9, it prints which colour is off and returns false — main() aborts with an error message asking the user to rescan. This catches most misclassifications before wasting time in the solver.

“Why don’t you use machine learning for colour classification?”

An ML model would need a training set of sticker images per lighting condition, per camera. For a college project it’s massive overkill — plain hue-thresholding + a click-to-fix UI hits >95% accuracy with zero training data. If I were shipping this as a consumer app, I’d train a small CNN (~1MB) as a fallback classifier and still keep the manual override.

PART 12 — MULTITHREADED SOLVER

Overview

Solver/IDASTarSolverMT.h is a parallel version of the IDA* solver. Enabled via the `--fast` command-line flag. Runs `min(18, hardware_concurrency())` threads (10 on my M4 Air).

Strategy: Root-Level Parallelism

IDA* explores an 18-branching tree. Instead of one thread walking the tree, I assign the 18 first-level moves across N worker threads round-robin: - Thread t handles first-moves $\{t, t + N, t + 2N, \dots\}$ for $t \in [0, N)$ - Each thread makes a private cube copy and runs a full recursive DFS on its subtree - **Zero shared mutable state during search** → no locks in the hot path

Coordination Between Threads

Only 3 shared variables, all lightweight:

```
std::atomic<bool> found(false);           // signal winner
std::atomic<int> globalNextBound(200);    // next IDA* iteration bound
std::mutex solutionMutex;                 // guards the solution vector
```

- Every DFS node checks `found.load(memory_order_relaxed)` — if another thread already won, exit early.
- When a thread finds a solution: `found.exchange(true)` — the *first* thread to flip the atomic wins the race; others see the flip and back off.
- After all threads finish an iteration without a solution, they merge their local `nextBound` into `globalNextBound` via `compare_exchange_weak` (lock-free CAS loop).

Why This Is Better Than a Work-Stealing Queue

A full work-stealing thread pool would give better load balance but add lock/coordination overhead. For a search space this branchy (18 root moves, deep tree per branch), the per-branch work is heavy enough that round-robin static assignment is close to optimal. Simpler + less contention wins.

Measured Speedup on M4 Air (10 cores)

Scramble depth	Single-threaded	Multithreaded (<code>--fast</code>)	Speedup
8 moves	0.4 s	0.3 s	~1.3× (too short)
12 moves	3 s	0.6 s	~5×
15 moves	~2 min	~15 s	~8×
Deep (18+)	~1 hour	~7 min	~9×

Never hits the theoretical 10× because: 1. Work is unevenly distributed — some first-moves have hard subtrees, others prune quickly. 2. First few IDA* iterations are too small to parallelize efficiently.

Interview Questions About Multithreading

“Why did you pick root-level parallelism over a shared work queue?”

A shared queue needs locking on every push/pop, and Rubik’s search is CPU-bound with millions of nodes per second — the lock would become the bottleneck. Root-level splitting has zero coordination during DFS, only atomics for the “solution found” signal. Simpler and faster.

“How does the winning thread stop the others?”

`std::atomic<bool> found` is checked at the top of every DFS call with `memory_order_relaxed`. When a thread finds a solution, it calls `found.exchange(true)`; the return value tells it whether it was the first to flip it (so it can safely store the solution under the mutex). Other threads see `found == true` on their next check and return `false` up the recursion stack, unwinding fast.

“Why `memory_order_relaxed`?”

The atomic is only used as a hint to abort search — we don’t need synchronisation of any other memory alongside it. The `found.exchange(true)` in the winning thread does need stronger ordering, but the frequent reads are on the hot path and relaxed is enough (worst case: a losing thread does a few extra useless nodes before noticing — negligible).

“What’s `compare_exchange_weak` and why weak?”

CAS operation: atomically read a value, and if it still equals expected, replace it with a new value. “Weak” is allowed to spuriously fail (return false even when the value matched) on some platforms — this makes it cheaper on ARM/M4. Since we’re in a retry loop anyway (`while (!globalNextBound.compare_exchange_weak(...))`), spurious failures cost nothing.

“Would a GPU speed this up further?”

Not really — Rubik’s search is highly divergent (each branch takes different paths), which is the worst case for SIMT GPU architectures. You’d get 100 threads all running different pruning conditions and completely different subtrees — kills warp coherence. CPU parallelism is the right fit.

“What if the machine has fewer than 10 cores?”

`std::thread::hardware_concurrency()` returns the actual core count at runtime; I `min(18, ...)` it and fall back to 1 if it returns 0 (rare). So the same binary scales from a 2-core laptop to a 32-core workstation without recompiling.